Project - Low Level Design
On
CI/CD pipeline for Node.js
Retail Application

Course Name: Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Kratin Verma	EN22CS301517
2.	Karishma Patidar	EN22CS301484
3.	Kanishka Patidar	EN22CS301480
4.	Jay Yadav	EN22CS301462

Group Name: Group 01D5

Project Number: DO-01

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Ritesh Joshi

Academic Year: 4th

Table of Contents

1.Introduction.

This document provides the detailed Low-Level Design (LLD) of the Retail DevOps Project. It explains internal components, workflows, deployment architecture, and implementation details of the system. The project focuses on automating software delivery using CI/CD practices.

1.1 Scope of the document.

This document includes:

- Detailed system architecture
- CI/CD workflow
- Application design
- Deployment architecture using Kubernetes
- API structure
- Component-level design

1.2 Intended Audience

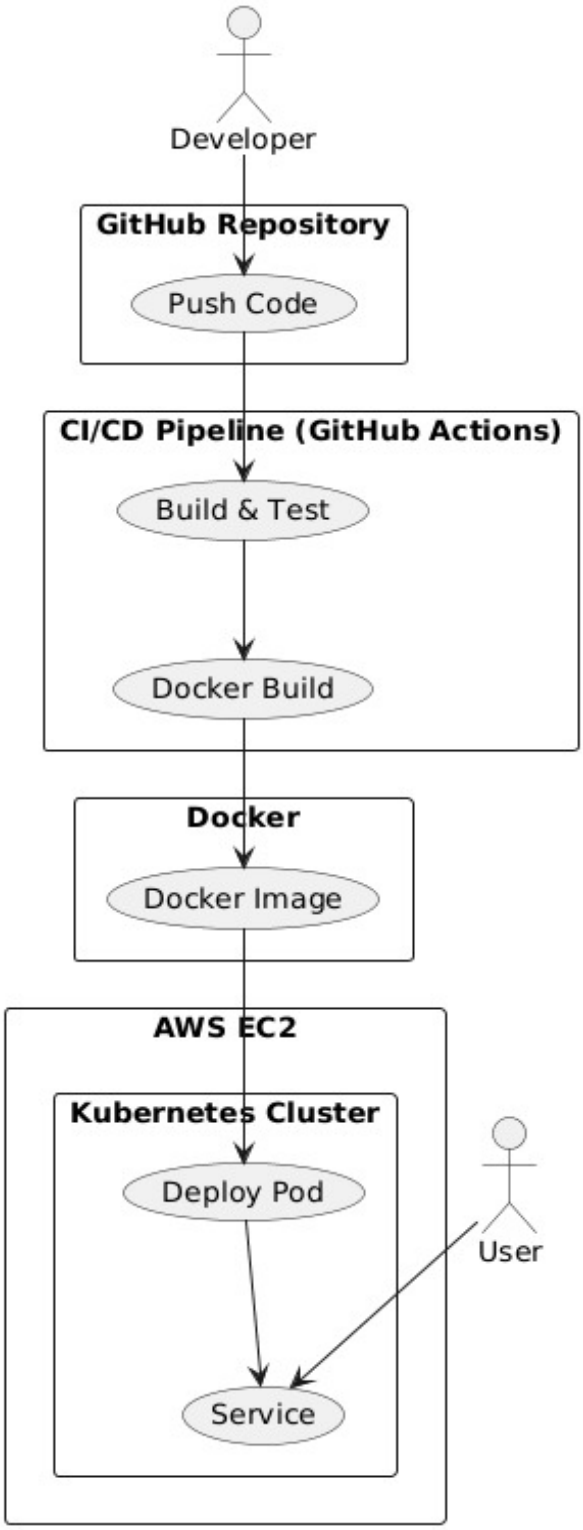
- Developers
- DevOps Engineers
- Faculty & Evaluators
- Project Mentors

1.3 System overview.

The system is a Retail Web Application deployed using DevOps practices.

Features:

- Automated CI/CD pipeline
- Containerized application
- Cloud-based deployment
- Scalable using Kubernetes


Developer
GitHub Repository
Push Code
CI/CD Pipeline (GitHub Actions)
Build & Test
Docker Build
Docker
Docker Image
AWS EC2
Kubernetes Cluster
Deploy Pod
Service
User

2.System Design.

Architecture Type:

- Layered Architecture
- Container-based deployment

Layers:

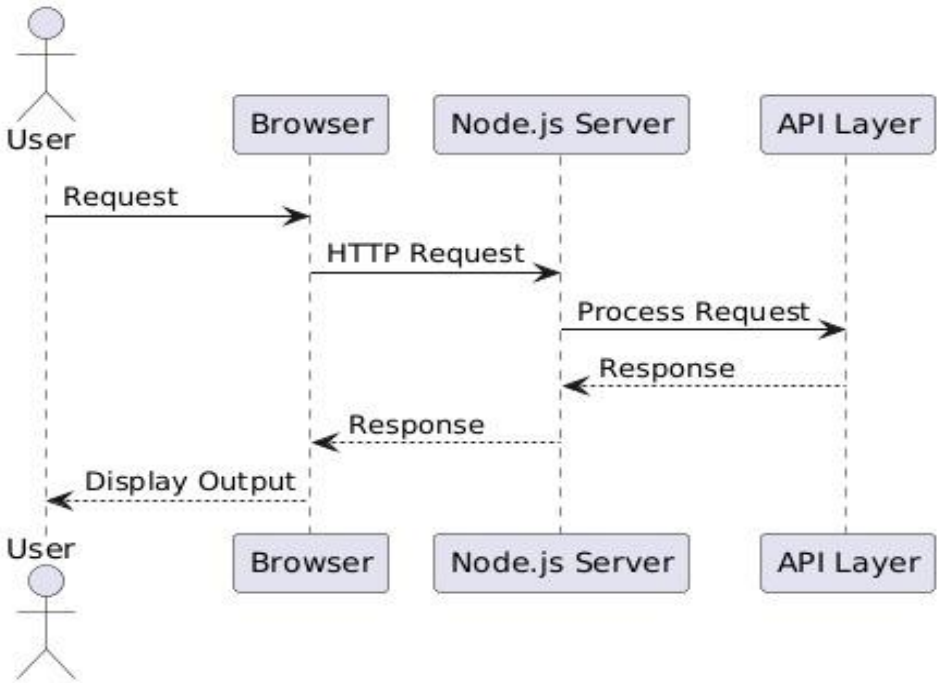
- Client Layer (Browser/User)
- Application Layer (Node.js)
- Container Layer (Docker)
- Orchestration Layer (Kubernetes)
- Infrastructure Layer (AWS EC2)

2.1 Application Design

The application is built using Node.js and follows REST architecture.

Modules:

- Server Module (Express.js)
- Routing Module
- API Module



2.2 Process Flow.

Steps:

i. Developer pushes code to repository

ii. CI/CD pipeline triggers

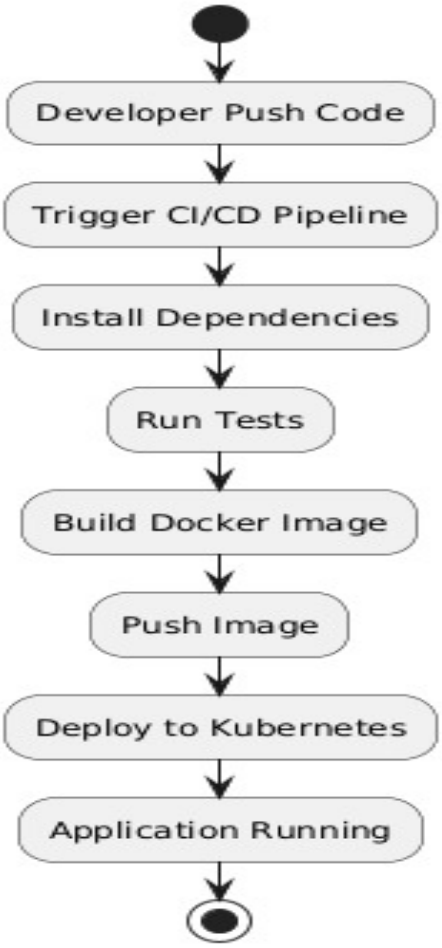
iii. Dependencies installed

iv. Tests executed

v. Docker image built

vi. Image deployed to Kubernetes

vii. Application exposed to users

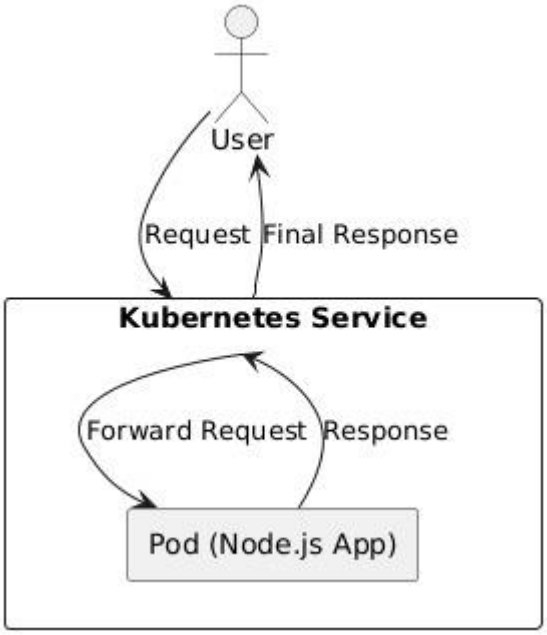


2.3 Information Flow.

The system processes user requests through Kubernetes services.

Flow:

i. User sends request
ii. Request reaches Kubernetes Service
iii. Service forwards to Pod
iv. Pod processes request
v. Response returned to user



2.4 Components Design

Components:

1. GitHub

- Stores code
- Triggers pipeline

2. CI/CD Pipeline

- Automates build & deployment

3. Docker

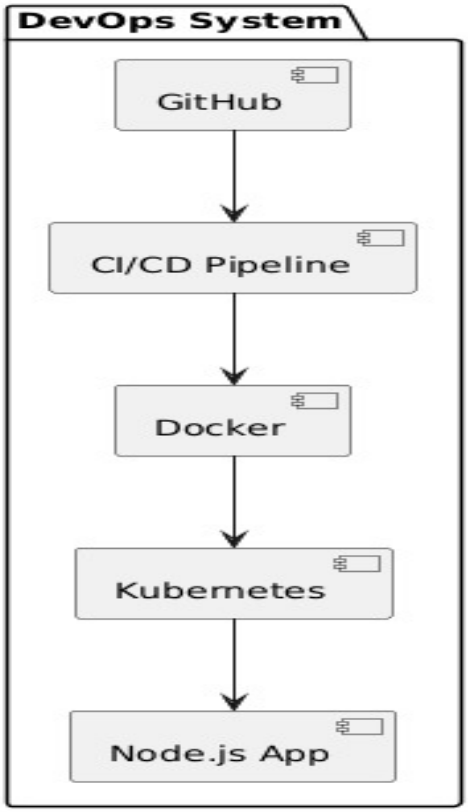
- Creates container images

4. Kubernetes

- Manages deployment
- Handles scaling

5. AWS EC2

- Hosts infrastructure



2.5 Key Design Considerations

a. Scalability: Kubernetes auto-scaling
b. Reliability: Container-based deployment
c. Security: AWS Security Groups
d. Automation: CI/CD pipeline
e. Performance: Lightweight containers

2.6 API Catalogue.

Base URL:

http://<EC2-IP>:NodePort

APIs:

1. Home API

GET /

Response: "Retail App Running"

2. Health API

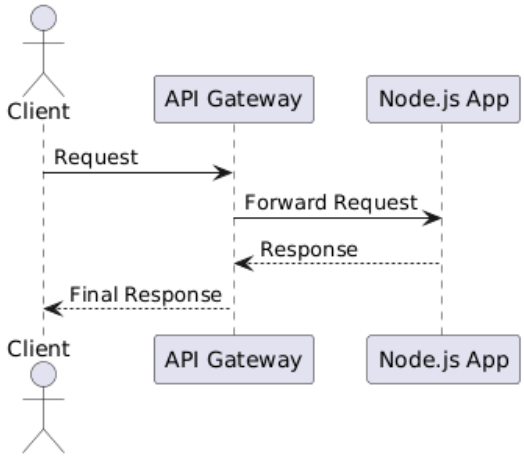
GET /health

Response: OK

3. Product API (Optional)

GET /products

POST /products



3.References

a. Node.js Documentation
b. Docker Documentation
c. Kubernetes Documentation
d. GitHub Actions Docs
e. AWS EC2 Documentation